Technische Dokumentation: Zod-Validierung im Anmeldeformular

Projekt: Anmeldeformular – Next.js
Datei: `AnFo_BB.tsx`
Technologien: Next.js, React, TypeScript und Zod

Inhaltsverzeichnis

1. Einführung in Zod
2. Warum wird Zod verwendet?
3. Import der Bibliothek
4. Definition des Validierungsschemas
5. Validierung der einzelnen Felder
6. Verwaltung der Formulardaten mit `useState`
7. Verwaltung der Fehlermeldungen
8. Eingaben verarbeiten und live validieren
9. Formular beim Absenden überprüfen
10. Fehlermeldungen aus Zod verarbeiten
11. Fehler im Formular anzeigen
12. Erfolgsmeldung bei gültigen Eingaben
13. Ablauf der gesamten Validierung
14. Wichtige Fachbegriffe
15. Grenzen der aktuellen Implementierung
16. Offizielle Dokumentation und weiterführende Links

1. Einführung in Zod

Zod ist eine Bibliothek für JavaScript und TypeScript, mit der sich Daten anhand festgelegter Regeln überprüfen lassen.

In diesem Projekt wird Zod verwendet, um die Eingaben eines Anmeldeformulars zu validieren. Dadurch wird kontrolliert, ob die eingegebenen Daten den erwarteten Anforderungen entsprechen.

Beispiele:

- Ein Vorname darf nicht leer sein.
- Eine Postleitzahl darf nur Ziffern enthalten.
- Eine E-Mail-Adresse muss ein gültiges Format besitzen.
- Ein Datum muss angegeben werden.

Zod ermöglicht es, diese Regeln zentral in einem sogenannten Schema zu definieren. Anschließend können die Eingaben gegen dieses Schema geprüft werden.

Die offizielle Dokumentation befindet sich unter:

https://zod.dev/

2. Warum wird Zod verwendet?

Ohne eine Validierungsbibliothek müssten die einzelnen Eingaben manuell überprüft werden. Für jedes Feld wären eigene Bedingungen und Fehlerbehandlungen notwendig.

Zod vereinfacht diesen Vorgang, da die Regeln übersichtlich definiert und wiederverwendet werden können.

Vorteile von Zod

Zentrale Regeln: Alle Validierungsregeln befinden sich im Objekt `formSchema`.

Wiederverwendbarkeit: Dasselbe Schema kann sowohl beim Absenden des Formulars als auch bei der Überprüfung einzelner Felder verwendet werden.

Verständliche Fehlermeldungen: Für viele Regeln können eigene Fehlermeldungen festgelegt werden.

TypeScript-Unterstützung: Zod lässt sich mit TypeScript kombinieren und unterstützt die Ableitung von Datentypen aus Schemas.

Kontrollierte Fehlerbehandlung: Mit `safeParse()` kann überprüft werden, ob Daten gültig sind, ohne dass bei einem Validierungsfehler eine Ausnahme ausgelöst wird.

Die Zod-Dokumentation erklärt diese grundlegenden Funktionen ausführlich:

https://zod.dev/basics

3. Import der Bibliothek

Am Anfang der Datei steht folgender Import:

```
import * as z from "zod";
```

Was bedeutet dieser Import?

`import` ist eine Anweisung von JavaScript beziehungsweise TypeScript, mit der Funktionen und andere Bestandteile einer Bibliothek eingebunden werden.

`zod` ist der Name des installierten Pakets.

`* as z` importiert die verfügbaren Exporte unter dem Namen `z`. Dadurch können die Funktionen der Bibliothek beispielsweise über `z.object()`, `z.string()` und `z.email()` verwendet werden.

Das Präfix z ist lediglich ein frei wählbarer Name für den Import. Es ist die übliche Schreibweise in der Zod-Dokumentation.

Damit der Import funktioniert, muss Zod im Projekt installiert sein:

```
npm install zod
```

Der Befehl installiert das Paket und trägt es normalerweise als Abhängigkeit in der Datei `package.json` ein.

4. Definition des Validierungsschemas

Das Schema wird im Code durch das Objekt `formSchema` definiert.

```
const formSchema = z.object({
  vorname: z.string().trim().min(1, 'Vorname ist erforderlich'),
  nachname: z.string().trim().min(1, 'Nachname ist erforderlich'),
  adresse: z.string().trim().min(1, 'Adresse ist erforderlich'),

  plz: z.string()
    .trim()
    .min(1, 'PLZ ist erforderlich')
    .regex(/^[0-9]+$/, 'PLZ darf nur Ziffern enthalten'),

  email: z.string()
    .trim()
    .min(1, 'E-Mail ist erforderlich')
    .pipe(z.email('Bitte gib eine gültige E-Mail ein')),

  datum: z.string().min(1, 'Datum ist erforderlich'),
});
```

4.1 Was ist ein Schema?

Ein Schema beschreibt, wie Daten aufgebaut sein müssen und welche Bedingungen sie erfüllen sollen.

In diesem Projekt legt das Schema fest, welche Eigenschaften die Formulardaten besitzen müssen und welche Regeln für diese Eigenschaften gelten.

4.2 Was macht `z.object()`?

`z.object()` erstellt ein Schema für ein Objekt.

Ein Objekt ist eine Datenstruktur, die mehrere Eigenschaften mit jeweils einem Wert zusammenfasst.

Die Formulardaten werden beispielsweise folgendermaßen gespeichert:

```
{
  vorname: 'Anna',
  nachname: 'Muster',
  adresse: 'Hauptstraße 10',
  plz: '1010',
  email: 'anna@example.com',
  datum: '2026-10-14'
}
```

`z.object()` ermöglicht es, für jede dieser Eigenschaften eigene Validierungsregeln festzulegen.

Da im Schema keine Eigenschaften als optional gekennzeichnet sind, werden alle sechs Felder bei der vollständigen Formularvalidierung erwartet.

4.3 Was bedeutet `const`?

`const` deklariert eine Variable, deren Zuweisung nicht erneut verändert werden kann.

Hier wird `formSchema` einmal definiert und anschließend an verschiedenen Stellen im Programm zur Validierung verwendet.

Das Schema muss nicht bei jeder Eingabe neu erstellt werden.

5. Validierung der einzelnen Felder

5.1 Vorname, Nachname und Adresse

Beispiel:

```
vorname: z.string()
  .trim()
  .min(1, 'Vorname ist erforderlich')
```

Die einzelnen Methoden haben folgende Bedeutung:

`z.string()`

Legt fest, dass der Wert ein Text sein muss.

`.trim()`

Entfernt führende und nachfolgende Leerzeichen bei der Validierung und Verarbeitung des Textwertes.

Beispiel:

`"  Anna  "` wird bei der Verarbeitung zu `"Anna"`.

Dadurch wird verhindert, dass ein Feld ausschließlich mit Leerzeichen als sinnvoll ausgefüllter Text akzeptiert wird.

`.min(1, 'Vorname ist erforderlich')`

Legt eine Mindestlänge von einem Zeichen fest. Ist der Text nach der Bereinigung leer, schlägt die Validierung fehl und die angegebene Fehlermeldung wird verwendet.

Die gleiche Logik wird für Nachname und Adresse angewendet. Lediglich der Name der Fehlermeldung unterscheidet sich.

5.2 Postleitzahl (PLZ)

Die Validierung der Postleitzahl lautet:

```
plz: z.string()
  .trim()
  .min(1, 'PLZ ist erforderlich')
  .regex(/^[0-9]+$/, 'PLZ darf nur Ziffern enthalten')
```

Die Postleitzahl wird als String und nicht als Zahl behandelt. Das ist sinnvoll, weil Postleitzahlen Kennzeichnungen und keine Rechenwerte sind. Außerdem können sie mit einer Null beginnen.

`.min(1, ...)`

Überprüft, ob mindestens ein Zeichen vorhanden ist.

`.regex(...)`

Prüft den Text anhand eines regulären Ausdrucks (Regular Expression, kurz Regex).

Der Ausdruck lautet:

```
/^[0-9]+$/
```

Er setzt sich aus mehreren Bestandteilen zusammen:

Bestandteil	Bedeutung
^	Beginn des Textes
[0-9]	Eine Ziffer zwischen 0 und 9
+	Mindestens ein Zeichen der vorherigen Gruppe
$	Ende des Textes

Damit werden nur Zeichenfolgen akzeptiert, die vollständig aus mindestens einer Ziffer bestehen.

Beispiele:

- `1010` – gültig
- `0123` – gültig
- `12A0` – ungültig
- `12 34` – ungültig
- Leerer Text – ungültig

Die Regel prüft nicht, ob die Postleitzahl tatsächlich existiert oder zum angegebenen Wohnort passt. Sie kontrolliert ausschließlich den Zeicheninhalt.

5.3 E-Mail-Adresse

Die E-Mail-Validierung lautet:

```
email: z.string()
  .trim()
  .min(1, 'E-Mail ist erforderlich')
  .pipe(z.email('Bitte gib eine gültige E-Mail ein'))
```

Auch hier werden mehrere Regeln kombiniert.

`z.string()`

Erwartet einen Textwert.

`.trim()`

Entfernt Leerzeichen am Anfang und am Ende.

`.min(1, 'E-Mail ist erforderlich')`

Stellt sicher, dass überhaupt eine Eingabe vorhanden ist.

`.pipe(...)`

Übergibt den erfolgreich geprüften Wert an ein weiteres Schema. Dadurch können mehrere Validierungsschritte hintereinander ausgeführt werden.

`z.email(...)`

Prüft, ob die eingegebene Zeichenfolge dem von Zod verwendeten E-Mail-Format entspricht.

Beispiele:

- anna@example.com – gültiges Format
- `anna.example.com` – ungültiges Format
- `anna@` – ungültiges Format
- Leerer Text – Pflichtfeldfehler

Die Prüfung bestätigt nicht, dass die E-Mail-Adresse existiert oder dass der Benutzer Zugriff auf das Postfach hat. Dafür wäre beispielsweise ein zusätzlicher Bestätigungsschritt notwendig.

Hinweis: Die Syntax `z.email()` entspricht der aktuellen Zod-4-API. Die Schreibweise mit `.pipe()` verbindet die Pflichtfeldprüfung mit der anschließenden Formatprüfung.

5.4 Datumsfeld

Die Regel für das Datum lautet:

```
datum: z.string().min(1, 'Datum ist erforderlich')
```

Das Datum wird aus dem HTML-Datumsfeld als String übernommen.

`.min(1, ...)` stellt sicher, dass das Feld nicht leer ist.

Da das Eingabefeld den Typ `date` verwendet, stellt der Browser eine passende Datumsauswahl zur Verfügung.

Das Schema prüft hier jedoch nur, ob ein Wert vorhanden ist. Es enthält keine zusätzliche Regel für einen bestimmten Zeitraum, beispielsweise ein Mindestalter oder ein Datum in der Zukunft.

6. Verwaltung der Formulardaten mit `useState`

Die Formulardaten werden mit Reacts `useState` verwaltet.

```
const [formData, setFormData] = useState({
  vorname: '',
  nachname: '',
  adresse: '',
  plz: '',
  email: '',
  datum: '',
});
```

Was ist `useState`?

`useState` ist ein React-Hook. Ein Hook ist eine React-Funktion, mit der eine Komponente bestimmte Funktionen wie Zustandsverwaltung verwenden kann.

Mit `useState` können Daten gespeichert werden, die sich während der Verwendung der Anwendung ändern.

Bedeutung der einzelnen Bestandteile

`formData`

Enthält die aktuellen Werte aller Eingabefelder.

`setFormData`

Ist die Funktion, mit der die gespeicherten Formulardaten aktualisiert werden.

`useState({...})`

Definiert den Anfangszustand. Zu Beginn sind alle Felder leer.

Wenn ein Benutzer einen Text eingibt, wird der entsprechende Wert aktualisiert. React rendert die Komponente anschließend bei Bedarf erneut, sodass der aktuelle Wert angezeigt wird.

Die Eingabefelder verwenden beispielsweise:

`value={formData.vorname}`

Dadurch wird der gespeicherte Wert als aktueller Inhalt des Eingabefeldes verwendet.

7. Verwaltung der Fehlermeldungen

Die Fehlermeldungen werden in einem eigenen Zustand gespeichert:

```
const [errors, setErrors] = useState<
  Partial<Record<keyof typeof formData, string>>
>({});
```

Dieser Ausdruck enthält mehrere TypeScript-Begriffe.

7.1 `errors`

`errors` ist ein Objekt, das die Fehlermeldungen für die einzelnen Felder enthält.

Ein Beispiel für einen möglichen Zustand:

```
{
  vorname: 'Vorname ist erforderlich',
  email: 'Bitte gib eine gültige E-Mail ein'
}
```

7.2 setErrors

Mit `setErrors` wird der Zustand der Fehlermeldungen aktualisiert.

Wenn ein Feld gültig ist, kann seine Fehlermeldung entfernt werden. Wenn es ungültig ist, wird die passende Meldung gespeichert.

7.3 keyof typeof formData

`typeof formData` beschreibt den Typ des Objekts `formData`.

`keyof` ermittelt die möglichen Eigenschaftsnamen dieses Typs.

Daraus ergibt sich sinngemäß:

```
'vorname' | 'nachname' | 'adresse' | 'plz' | 'email' | 'datum'
```

Damit wird festgelegt, dass die Fehlerverwaltung nur die bekannten Feldnamen verwenden soll.

7.4 Record<..., string>

`Record` ist ein TypeScript-Hilfstyp.

Er beschreibt ein Objekt, bei dem die zulässigen Schlüssel festgelegt sind und deren Werte jeweils Strings sein müssen.

7.5 Partial<...>

`Partial` macht die Eigenschaften eines Typs optional.

Das ist hier sinnvoll, weil nicht jedes Feld jederzeit einen Fehler besitzen muss.

7.6 {}

Das leere Objekt ist der Anfangszustand der Fehlerverwaltung. Zu Beginn wurden noch keine Fehlermeldungen gespeichert.

8. Eingaben verarbeiten und live validieren

Die Funktion handleChange wird ausgeführt, sobald sich der Wert eines Eingabefeldes verändert.

```
const handleChange = (
  e: React.ChangeEvent<HTMLInputElement>
) => {
  const { name, value } = e.target;
  const field = name as keyof typeof formData;

  const updatedFormData = {
    ...formData,
    [field]: value,
  };

  setFormData(updatedFormData);

  const result = formSchema.shape[field].safeParse(value);

  setErrors((prev) => ({
    ...prev,
    [field]: result.success
      ? undefined
      : result.error.issues[0].message,
  }));
};
```

8.1 Der Event-Handler

```
e: React.ChangeEvent<HTMLInputElement>
```

e steht für das Ereignis (Event), das beim Ändern des Eingabefeldes ausgelöst wird.

`React.ChangeEvent<HTMLInputElement>` ist ein TypeScript-Typ, der beschreibt, welches Ereignis und welches Element verarbeitet werden.

Dadurch weiß TypeScript, dass auf die Eigenschaften des geänderten HTML-Eingabefeldes zugegriffen wird.

8.2 `e.target`

`e.target` bezeichnet das Element, von dem das Ereignis ausgelöst wurde.

Die Anweisung

```
const { name, value } = e.target;
```

liest zwei Eigenschaften aus:

- `name`: den Namen des Eingabefeldes, zum Beispiel `vorname`.
- `value`: den aktuellen Inhalt des Feldes.

Die geschweiften Klammern werden hier für die sogenannte Destrukturierung verwendet. Damit können mehrere Eigenschaften direkt aus einem Objekt entnommen werden.

8.3 Type Assertion mit as

```
const field = name as keyof typeof formData;
```

Mit `as` wird TypeScript mitgeteilt, dass `name` als einer der bekannten Feldnamen behandelt werden soll.

Das ist eine Type Assertion. Sie verändert den Wert zur Laufzeit nicht und überprüft auch nicht automatisch, ob der Name tatsächlich gültig ist.

Die Zuordnung funktioniert deshalb zuverlässig, weil die `name`-Attribute der Eingabefelder mit den Eigenschaften aus `formData` übereinstimmen.

8.4 Aktualisierung der Formulardaten

```
const updatedFormData = {
  ...formData,
  [field]: value,
};
```

Der Spread-Operator `...formData` übernimmt die bisherigen Eigenschaften des Objekts.

`[field]: value` überschreibt anschließend den Wert des gerade bearbeiteten Feldes.

So bleiben alle anderen Eingaben erhalten.

Danach speichert

```
setFormData(updatedFormData);
```

das aktualisierte Objekt im React-Zustand.

8.5 Einzelnes Feld mit Zod überprüfen

```
const result = formSchema.shape[field].safeParse(value);
```

Dieser Ausdruck greift über `.shape` auf das Schema des jeweiligen Feldes zu.

Beispielsweise wird bei `name="plz"` das Schema der Postleitzahl verwendet.

Anschließend prüft `safeParse(value)`, ob der aktuelle Feldwert den Regeln entspricht.

Das Ergebnis enthält entweder:

- `success: true`, wenn die Eingabe gültig ist.
- `success: false`, wenn die Eingabe ungültig ist, einschließlich Informationen über die Fehler.

Die Methode `safeParse()` löst bei einem normalen Validierungsfehler keine Ausnahme aus. Das Ergebnis kann direkt mit einer Bedingung verarbeitet werden.

8.6 Fehlermeldung aktualisieren

```
setErrors((prev) => ({
  ...prev,
  [field]: result.success
    ? undefined
    : result.error.issues[0].message,
}));
```

Hier wird der Zustand der Fehlermeldungen aktualisiert.

`prev` enthält den bisherigen Zustand von `errors`.

Der Spread-Operator `...prev` übernimmt die bisherigen Fehlermeldungen der anderen Felder.

Anschließend wird nur die Fehlermeldung des geänderten Feldes aktualisiert.

Der Ausdruck

```
result.success
  ? undefined
  : result.error.issues[0].message
```

verwendet den ternären Operator. Er ist eine Kurzform für eine einfache Bedingung.

- Wenn `result.success` wahr ist, wird `undefined` gespeichert. Dadurch wird die Fehlermeldung des Feldes entfernt.
- Wenn `result.success` falsch ist, wird die erste Fehlermeldung von Zod gespeichert.

`issues` ist eine Liste der Validierungsprobleme. Mit `[0]` wird das erste Problem ausgewählt, und `.message` liefert den dazugehörigen Meldungstext.

8.7 Warum erscheint die Fehlermeldung erneut?

Die Funktion entfernt Fehlermeldungen nicht mehr pauschal bei jeder Eingabe. Stattdessen überprüft sie den aktuellen Wert erneut.

Beispiel für den Vornamen:

1. Das Feld ist leer und wird validiert.
2. Der Benutzer gibt Anna ein. Der Wert ist gültig und die Fehlermeldung wird entfernt.
3. Der Benutzer löscht den Text vollständig.
4. Zod überprüft den leeren Wert erneut.
5. Die Regel `.min(1, 'Vorname ist erforderlich')` schlägt fehl.
6. Die Fehlermeldung wird wieder angezeigt.

Damit bleibt die Fehleranzeige mit dem aktuellen Zustand der Eingabe synchron.

Hinweis: In der aktuellen Implementierung wird die Live-Validierung bereits bei der ersten Änderung eines Feldes aktiviert. Deshalb können Fehlermeldungen auch schon während der ersten Eingabe erscheinen. Wenn die Fehler erst nach dem ersten Absendeversuch angezeigt werden sollen, ist eine zusätzliche Zustandsvariable erforderlich.

9. Formular beim Absenden überprüfen

Die Funktion `handleSubmit` verarbeitet das Absenden des gesamten Formulars.

```
const handleSubmit = (
  e: React.FormEvent<HTMLFormElement>
) => {
  e.preventDefault();

  const result = formSchema.safeParse(formData);

  // Weitere Fehlerbehandlung ...
};
```

9.1 `React.FormEvent<HTMLFormElement>`

Dieser Typ beschreibt ein Ereignis, das durch ein HTML-Formular ausgelöst wird.

9.2 `e.preventDefault()`

Normalerweise versucht ein HTML-Formular beim Absenden, seine Daten nach den Standardregeln des Browsers zu übermitteln.

`preventDefault()` verhindert dieses Standardverhalten.

In diesem Projekt übernimmt React die Verarbeitung des Formulars, weshalb das Formular nicht auf herkömmliche Weise abgesendet werden soll.

9.3 Vollständige Validierung mit `safeParse()`

```
const result = formSchema.safeParse(formData);
```

Hier wird nicht nur ein einzelnes Feld überprüft, sondern das gesamte Formularobjekt.

Zod kontrolliert sämtliche Regeln aus `formSchema`.

Wenn eine oder mehrere Regeln verletzt werden, enthält das Ergebnis die entsprechenden Validierungsprobleme.

Wenn alle Regeln erfüllt sind, ist `result.success` wahr.

10. Fehlermeldungen aus Zod verarbeiten

Wenn die vollständige Validierung fehlschlägt, wird folgender Abschnitt ausgeführt:

```
if (!result.success) {
  const newErrors: Partial<
    Record<keyof typeof formData, string>
  > = {};

  for (const issue of result.error.issues) {
    const field = issue.path[0] as keyof typeof formData;

    if (!newErrors[field]) {
      newErrors[field] = issue.message;
    }
  }
```

```
  setErrors(newErrors);
  return;
}
```

10.1 `if (!result.success)`

`if` führt einen Codeblock nur aus, wenn die Bedingung erfüllt ist.

Das Ausrufezeichen `!` bedeutet hier logische Negation.

Die Bedingung ist also wahr, wenn die Validierung nicht erfolgreich war.

10.2 `newErrors`

```
const newErrors = {};
```

Hier wird ein neues Objekt vorbereitet, in dem die aktuellen Fehler gesammelt werden.

Das ist sinnvoll, weil beim Absenden alle Felder überprüft werden. Die Fehlerliste soll deshalb den aktuellen Validierungsstand des gesamten Formulars widerspiegeln.

10.3 `for...of`

```
for (const issue of result.error.issues)
```

Diese Schleife verarbeitet jedes Validierungsproblem nacheinander.

Wenn beispielsweise Vorname und E-Mail ungültig sind, können beide Fehler in der Liste enthalten sein.

10.4 `issue.path`

```
const field = issue.path[0] as keyof typeof formData;
```

`issue.path` beschreibt den Pfad zu der Eigenschaft, bei der der Fehler aufgetreten ist.

Bei einem einfachen Formularobjekt enthält der erste Eintrag normalerweise den Feldnamen, zum Beispiel `email` oder `plz`.

10.5 `issue.message`

`issue.message` enthält die Fehlermeldung, die durch die verletzte Validierungsregel erzeugt wurde.

Beispielsweise:

`Bitte gib eine gültige E-Mail ein`

10.6 `if (!newErrors[field])`

Diese Bedingung prüft, ob für das betreffende Feld bereits eine Meldung gespeichert wurde.

Wenn noch keine Meldung vorhanden ist, wird die aktuelle Meldung eingetragen. Dadurch bleibt pro Feld zunächst die erste gefundene Meldung erhalten.

10.7 `setErrors(newErrors)` und `return`

`setErrors(newErrors)` übernimmt die neu erstellte Liste der Fehler.

`return` beendet anschließend die Funktion. Dadurch wird verhindert, dass die Erfolgsmeldung angezeigt wird, obwohl noch ungültige Eingaben vorhanden sind.

11. Fehlermeldungen im Formular anzeigen

Im JSX werden die Fehlermeldungen direkt unter den entsprechenden Eingabefeldern angezeigt.

Beispiel:

```
<input
  className="inputfielsec"
  placeholder="Vorname"
  name="vorname"
  value={formData.vorname}
  onChange={handleChange}
/>

{errors.vorname && (
  <p className="error">{errors.vorname}</p>
)}
```

Erklärung

`value={formData.vorname}`

Der Inhalt des Eingabefeldes wird vom React-Zustand gesteuert.

`onChange={handleChange}`

Bei jeder Änderung wird die Funktion `handleChange` aufgerufen.

`errors.vorname && (...)`

Das ist eine bedingte Darstellung in JSX.

Der Ausdruck prüft, ob für den Vornamen eine Fehlermeldung vorhanden ist. Nur dann wird der folgende Absatz gerendert.

`<p className="error">`

Der Absatz zeigt den Text der Fehlermeldung an. Über die CSS-Klasse `error` kann das Aussehen der Meldung gestaltet werden.

Diese Technik wird für alle sechs Felder verwendet.

12. Erfolgsmeldung bei gültigen Eingaben

Wenn die vollständige Validierung erfolgreich ist, wird folgender Code ausgeführt:

```
setErrors({});
alert('Formular erfolgreich ausgefüllt!');
```

`setErrors({})`

Das leere Objekt entfernt alle gespeicherten Fehlermeldungen aus dem React-Zustand.

`alert(...)`

`alert()` öffnet ein Meldungsfenster des Browsers und zeigt den angegebenen Text an.

In diesem Fall erscheint:

Formular erfolgreich ausgefüllt!

Wichtig: Die Meldung bestätigt nur, dass die Eingaben die definierten Validierungsregeln erfüllen. Sie bedeutet nicht, dass die Formulardaten gespeichert oder an einen Server übertragen wurden.

13. Ablauf der gesamten Validierung

Der Ablauf lässt sich in zwei Bereiche unterteilen.

Während der Eingabe

1. Der Benutzer verändert ein Eingabefeld.
2. `handleChange` liest Feldname und Wert aus.
3. `formData` wird aktualisiert.
4. Das Schema des geänderten Feldes wird mit `safeParse()` überprüft.
5. Bei einem gültigen Wert wird die Fehlermeldung entfernt.
6. Bei einem ungültigen Wert wird die passende Fehlermeldung gespeichert und angezeigt.

Beim Absenden

1. Der Benutzer klickt auf „Absenden“.
2. `handleSubmit` verhindert das standardmäßige Formularverhalten.
3. `formSchema.safeParse(formData)` überprüft alle Felder.
4. Bei ungültigen Daten werden die Fehler gesammelt und angezeigt.
5. Bei gültigen Daten werden alle Fehlermeldungen entfernt.
6. Eine Erfolgsmeldung wird angezeigt.

Damit werden sowohl einzelne Felder während der Eingabe als auch das gesamte Formular beim Absenden überprüft.

14. Wichtige Fachbegriffe

Begriff	Bedeutung
Bibliothek	Wiederverwendbarer Code, der Funktionen für bestimmte Aufgaben bereitstellt

Schema	Beschreibung der erwarteten Datenstruktur und ihrer Regeln
Validierung	Überprüfung, ob Daten bestimmte Anforderungen erfüllen
Pflichtfeld	Ein Feld, für das eine Eingabe erforderlich ist
String	Zeichenfolge beziehungsweise Text
Objekt	Datenstruktur mit Eigenschaften und zugehörigen Werten
Methode	Funktion, die über ein Objekt oder einen bestimmten Wert aufgerufen wird
Regex	Regulärer Ausdruck zum Prüfen von Zeichenfolgen
Event	Ereignis, beispielsweise eine Änderung eines Eingabefeldes
Event-Handler	Funktion, die auf ein Ereignis reagiert
Zustand (State)	In React gespeicherte Daten, die sich während der Nutzung ändern können
Hook	React-Funktion für bestimmte Funktionen wie Zustandsverwaltung
TypeScript	Erweiterung von JavaScript mit einem statischen Typsystem
JSX	Syntax, mit der HTML-ähnliche Elemente in React-Code geschrieben werden
`safeParse()`	Validiert Daten und gibt ein Ergebnisobjekt zurück, statt bei einem normalen Validierungsfehler eine Ausnahme auszulösen
`issues`	Liste der bei einer Validierung festgestellten Probleme
`issue.path`	Pfad zur Eigenschaft, bei der ein Validierungsproblem aufgetreten ist
`issue.message`	Text der zugehörigen Fehlermeldung
`undefined`	Wert, der hier verwendet wird, um eine nicht vorhandene Fehlermeldung darzustellen
`return`	Beendet die Ausführung einer Funktion
Spread-Operator `...`	Übernimmt Eigenschaften eines Objekts oder Elemente einer anderen Datenstruktur
`Partial`	TypeScript-Hilfstyp, der die Eigenschaften eines Typs optional macht
`Record`	TypeScript-Hilfstyp für Objekte mit festgelegten Schlüssel- und Werttypen

15. Grenzen der aktuellen Implementierung

Die aktuelle Umsetzung überprüft die Eingaben im Browser. Das ist für die Rückmeldung an den Benutzer sinnvoll, ersetzt aber keine serverseitige Validierung.

Folgende Einschränkungen bestehen:

- Die E-Mail-Prüfung bestätigt nicht, dass eine Adresse tatsächlich existiert.

- Die PLZ-Prüfung kontrolliert nur, ob die Eingabe aus Ziffern besteht.
- Das Datum wird lediglich auf einen nicht leeren Wert überprüft.
- Die Eingaben werden nicht dauerhaft gespeichert.
- Die Daten werden nicht an einen Server übermittelt.
- Clientseitige Prüfungen können umgangen werden. Bei einer späteren Übermittlung müssen die Daten daher zusätzlich serverseitig validiert werden.

Für eine produktive Anwendung sollten die Daten vor einer Verarbeitung auf dem Server erneut geprüft werden.

16. Offizielle Dokumentation und weiterführende Links

Die folgenden Quellen erklären die verwendeten Funktionen und bieten weitere Beispiele.

Zod

Offizielle Website:
https://zod.dev/

Basic Usage – Grundlagen:
https://zod.dev/basics

Erklärt die Definition von Schemas, die Datenvalidierung mit `parse()` und `safeParse()` sowie die Verarbeitung von Validierungsfehlern.

API Reference – Schemas und Methoden:
https://zod.dev/api

Enthält die Referenz zu `z.object()`, `z.string()`, `.trim()`, `.min()`, `.regex()`, `z.email()` und weiteren Validierungsfunktionen.

React

React Hooks:
https://react.dev/reference/react

Dokumentation zu React-Hooks, darunter `useState`.

Input-Komponenten:
https://react.dev/reference/react-dom/components/input

Erklärt die Verwendung von Eingabefeldern und kontrollierten Komponenten.

TypeScript

TypeScript Handbook:
https://www.typescriptlang.org/docs/handbook/intro.html

Grundlagen zu Typen, Objekten, Funktionen und TypeScript-Ausdrücken.

Fazit

Zod übernimmt in diesem Projekt die zentrale Prüfung der Formulareingaben. Durch die Definition eines Schemas können Regeln für alle Felder an einer Stelle festgelegt werden.

Die Kombination aus `safeParse()`, Reacts `useState` und der Funktion `handleChange` ermöglicht es, Eingaben während der Bearbeitung zu überprüfen und Fehlermeldungen automatisch an den aktuellen Zustand der Felder anzupassen.

Beim Absenden überprüft `handleSubmit` das gesamte Formular erneut. Nur wenn alle definierten Regeln erfüllt sind, wird die Erfolgsmeldung angezeigt.

Die Implementierung verbessert damit die Eingabeprüfung und Benutzerfreundlichkeit des Anmeldeformulars, stellt aber noch keine dauerhafte Speicherung oder serverseitige Verarbeitung der Daten bereit.